

01 - How to implement a SWC to read/write DID in FACEasy (Classic AutoSAR) ?

Introduction

This page is mainly addressed to SW developers but it can also help system&sw architecture understand the difficulty of implementing a DID in FACEasy (Classic AutoSAR).

Through this document, you will learn by example how to use configured DIDs (DataIdentifier) and how to implement the Read and Write data operations.

Assumptions

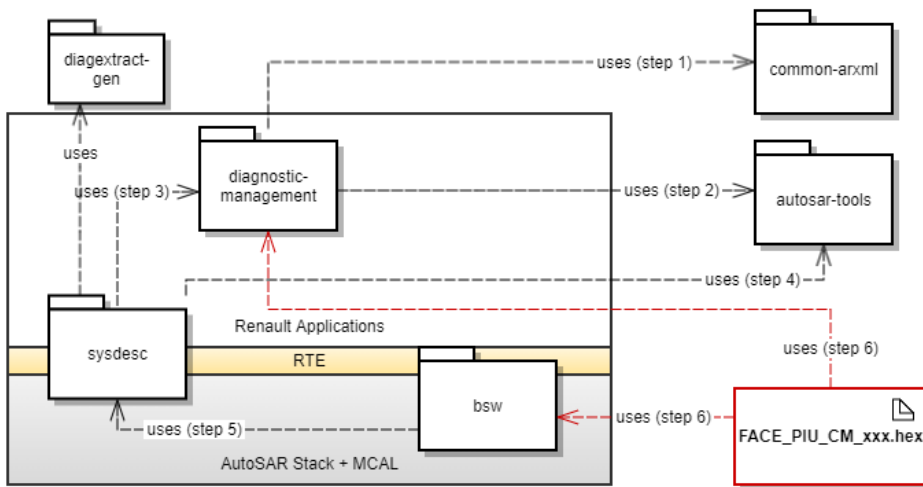
- Standard DIDs (E.g VIN, SystemSupplierECUSoftwareNumber, ...) are already listed in the DiagExtract_export.arxml
- We use the package autosar-tools (>= 18.0)
- We have already created a package to host our demo application
- You already know the definition of DID and have an overview of the diagnostic stack including the DCM (Diagnostic Communication Manager) in Classic AutoSAR.
- We chose the VIN (Vehicle Identification Number) as an example because it is a Read Write data.



The implementation given in this document does not cover the requirements of the VIN DID.

Project's structure

FACE project uses a set of packages including tools, Renault applications (code and models) and AutoSAR stack (code and basic configuration).



In this example, we show only the packages we use directly. Of course, the image depends on other packages which are not represented in this diagram for readability purpose.

Let's take a look at this diagram, we have our package diagnostic-management holding our demo application that implements the VIN Read and Write DID (discussed later).

(step 1) Our package depends on common-arxml that offers the interface files (AutoSAR XML aka ARXML) that describes for example the platform types and the types defined by the DCM (E.g Dcm_OpStatusType)

(step 2) Once the application's model (rncid) is ready, we use autosar-tools to compile rncid code into standard ARXML files.

(step 3) The DiagExtract_export.arxml file will be used by the sysdesc (system description) to map the diagnostic service needs of our application. In our case, each did read operation (function) is invoked by the OS (Operating System) thanks to a specific event mapped to this function.

(step 4) Integrator/developer should map events and DIDs in the sysdesc project (discussed later) and use autosar-tools (part of the FACE's toolchain) to generate the system description files.

(step 5) Based on the whole system description, the BSW tools (E.g EB Tresos) will generate RTE and BSW code (header and source files).

(step 6) Generated BSW code will be compiled with our software component's source, linked together and we get the binary image.

(Step 1 & Step 2) - Implement the demo application

Every software components has at least 1xrndll , 1xC, 1xCMakeLists.txt:

```
diagnostic-management/demo/swcReadWriteDid/  
├── CMakeLists.txt  
├── itfDid.rndll  
└── src  
    ├── swcReadWriteDid.c  
    └── swcReadWriteDid.rndll
```

You can use the following minimalist CMakeLists.txt template and modify the name of the component.

CMakeList

```
# import build scripts  
find_package(BuildScripts REQUIRED CONFIG)  
  
# import common ARXML  
find_package(CommonArxml REQUIRED CONFIG)  
  
include(AddCpGenerate)  
include(AddCpLibrary)  
  
# Include tests to avoid build dependency errors  
include(AddUnitTest)  
include(AddSwcTest)  
include(AddSwcTraceability)  
  
set(SWCNAME swcReadWriteDid)  
set(CP_PACKAGENAME ${SWCNAME})  
  
# Generate cp arxml for a swc based on rndll/fidl  
add_cp_generate(${CP_PACKAGENAME}  
    ${SWCNAME}.rndll  
    SWCNAME ${SWCNAME}  
    IMPORTS  
        # import basic elements (types, units and swaddr methods)  
        ${COMMON_ARXML_DIR}/type  
        ${COMMON_ARXML_DIR}/unit  
        ${COMMON_ARXML_DIR}/swaddr  
  
    INCLUDES  
        ${COMMON_ARXML_DIR}/itf  
        ${CONAN_INTERFACESARXML_ROOT}/lib/fidl  
)  
  
# Create a library which includes the source files  
# The last line in this CMakeLists.txt is commented, as the .c file does not exist yet.  
# add_cp_library(${CP_PACKAGENAME} src/swcReadWriteDid.c EXPORTEDPATHS src RTE)
```

Don't forget to include the subdirectory containing your CMakeLists.txt in the top level ~/diagnostic-management/CMakeLists.txt as follow

CMakeList

```
...  
add_subdirectory (demo/swcReadWriteDid)  
...
```

Then you need to create the rndll file where you can define the ports, the runnables, etc

Before we start the implementation, we need to know which interface we should use to implement the VIN. The definition of Diagnostic objects like DIDs can be found in this file: https://gitlabee.dt.renault.com/swlabs/face/ez1/diagextract-gen/-/blob/master/devices/piu-cm-production/ Diag_RN_Candela_v2.3.xlsx

The latter is used by **CandelaStudio** tool to generate a **DiagExtract ARXML** file which is the input file for developers and testers.

The DiagExtract ARXML and CDD files can be found in the artifacts of the CI Job **candela-gen:piu-cm-production** and in the CONAN package **piu-cm-production-diag-extract**

The VIN is a Read/Write DID. We can choose the Client/Server interface to describe its interface:

itfDid.rnidl

```
package face.swcReadWriteDid

import "itf_AutosarDcm.arxml"

typedef UInt8[17] F190_VINType
interface clientServer serviceBsw F190_VIN {
    op ReadData {
        in {Dcm_OpStatusType OpStatus}
        out {F190_VINType Data}
        error {
            E_NOT_OK = 1
            DCM_E_PENDING = 10
        }
    }
    op WriteData {
        in {
            F190_VINType Data
            Dcm_OpStatusType OpStatus
        }
        out {Dcm_NegativeResponseCodeType ErrorCode}
        error {
            E_NOT_OK = 1
            DCM_E_PENDING = 10
        }
    }
}
```

First of all, We import the "itf_AutosarDcm.arxml" because we need the definition of "Dcm_OpStatusType" and "Dcm_NegativeResponseCodeType". The VIN interface defines 2 CLIENT-SERVER operations. The first one is called ReadData with 1 input, 1 output and 2 possible errors. The second one is called WriteData with 2 inputs, 1 output and 2 possible errors.

Once the interface is defined in the rnidl, we can use it to create our VIN port, ReadData and WriteData runnables and define the diagnostic service need used later to map our application DIDs ports to the DCM:

swcReadWriteDid.rnidl

```
package face

import "itfDid.rnidl"

// SWC ID = RWD
swcAppl swcReadWriteDid {
    version {major 0 minor 1}

    property (multiInstance = false)
        pport F190_VIN          PP_F190_VIN

        // This runnable returns the VIN DID's value
    runnable RWD_F190_VIN_ReadData {
        event op PP_F190_VIN.ReadData
    }

    // This runnable sets the VIN DID's value
    runnable RWD_F190_VIN_WriteData {
        event op PP_F190_VIN.WriteData
    }

    // Service Need for F190_VIN did
    serviceNeeds diagValue F190_VIN {
        port DataServices PP_F190_VIN
        property (read, write)
    }
}
```

Now, it's time to show how to generate the ARXML files and the C template code from these rnidl files. First of all, you need to add "autosar-tools" and "common-arxml" dependencies to your conandata.yml:

conandata.yml

```
build_requirements:
    ...
    - build-scripts/9.7.0@face/stable
    - autosar-tools/21.5.10@face/stable
    - common-arxml/5.2.0@face/stable
    - bsw-cabin-master-interfaces/4.3.0@face/stable
    - interfaces-arxml/14.34.0@face/stable
    - gtest-multilib/1.0.2@face/stable
```

Then, we should run these commands in a terminal (Please make sure that docker is installed and configured as described in the README of your project):

```
cd ~/diagnostic-management
. setenv.sh          # Set up the environment and enable commands below
clean .              # remove the build directory if exists
install .            # fetch and install dependencies
build .              # compile rnidl files, generate arxml files, C template code and RTE header files
```

Finally, we need to implement our ReadData and WriteData functions on C file skeleton generated from the Runnables in swcReadWriteDid.rnidl. To do so, we copy the template code from `build/demo/swcReadWriteDid/swcReadWriteDid/testenv/rte/swcReadWriteDid.c` to `demo/swcReadWriteDid/src/swcReadWriteDid.c` and we fill the blanks !!!

swcReadWriteDid.c

```
#include <Rte_swcReadWriteDid.h>

FUNC(Std_ReturnType /*@alt void*/ , RTE_CODE) RWD_F190_VIN_ReadData (Dcm_OpStatusType OpStatus, /*@out*/ P2VAR
(uint8, AUTOMATIC, RTE_APPL_DATA) Data)
{
    /* Ignore OpStatus argument */
    (void)OpStatus;

    for (int i = 0; i < 17; i++) {
        Data[i] = 0xAB;
    }

    return RTE_E_OK;
}

FUNC(Std_ReturnType /*@alt void*/ , RTE_CODE) RWD_F190_VIN_WriteData (/*@in*/ P2CONST(uint8, AUTOMATIC,
RTE_APPL_DATA) Data, Dcm_OpStatusType OpStatus, /*@out*/ P2VAR(Dcm_NegativeResponseType, AUTOMATIC,
RTE_APPL_DATA) ErrorCode)
{
    /* Ignore OpStatus and Data argument */
    (void)OpStatus;
    (void)Data;

    /* It's a just an example */

    *ErrorCode = DCM_E_POSITIVERESPONSE;

    return RTE_E_OK;
}
```

After this point, uncomment the line `add_cp_library(${CP_PACKAGENAME} src/swcReadWriteDid.c EXPORTEDPATHS src RTE)` in `demo /swcReadWriteDid/CMakeLists.txt` and pack the description.

```
package .
```

(Step 3 & Step 4 & step 5) - Instantiate the application, connect it to DCM services and generate the system's code

The project where we instantiate applications in FACE is called `device-cm-<production/acceptance/etc ...>`. In our case, we will generate a production image. So we select the `device-cm-production`. We clone this project then we reference our built package in the `sysdesc/conandata.yml` and thus our application will be available and ready for use.

conandata.yml

```
build_requirements:
    ...
    # use latest package version of our application
    # the package version is printed by the command "package ." invoked in the diagnostic-management project
    # Let's take a look at the "package ." output in diagnostic-management project:
    # We get this log "Creating alias diagnostic-management/latest@topic/faceswp-3919-demo-did" and we deduced
    the name of the package
    # Also if you branch is local you can add the full name instead of alias, it can also be obtained from
    package output
    # (e.g. diagnostic-management/1.3.61-pre.2.g85d73@dev/faceswp-3919-demo-
    did#5e92a3a1899e3f4f79173b1682485f52)
    - diagnostic-management/latest@topic/faceswp-3919-demo-did
    - piu-cm-production-diag-extract/0.1.10@face/stable
    ...
```

Now we reference our application in `device-cm-production/sysdesc/device/CMakeLists.txt` and instantiate the application in `device-cm-production/sysdesc /device/swcCabinMaster.rnidl` as follow

CMakeList

```
...
add_cp_ecu(${PROJECT_NAME} ${PROJECT_NAME}.rncdl "bswconfig/CpEcuc_rteEvtMap.rncdl"
  SWCPACKAGES
    ...
    "swcReadWriteDid"
    ...
)
...
```

swcCabinMaster.rncdl

```
...
// swc swcReadWriteDid swcReadWriteDid
SWC swcReadWriteDid swcReadWriteDid
...
```

Then we need to map our VIN DID of the swcReadWriteDid to the DID from the DiagExtract in [device-cm-production/.../sysdesc/device/mapdiagPiuCabinMaster.rncdl](#)

swcBswCabinMaster.rncdl

```
...
// Map all ServiceNeeds onto Diagnostic elements
mapping {
    ...
    // Map VIN DID
    // NB: Since the Read and Write belong to the same interface/service need
    // mapping only one of these operations (Read/Write) is enough to let the BSW tool
    // guess the mapping of the second operation
    // You can also map both of them but the ArxmGen may not accept this change for limitation
issues
    // serviceNeeds swcReadWriteDid.F190_VIN F190_VIN_Write
    serviceNeeds swcReadWriteDid.F190_VIN F190_VIN_Read
    ...
}
...
```

The last step is to map the events of our SWC to the Diagnostic Task in `device-cm-production/sysdesc/device/bswconfig/CpEcuc_rteEvtMap.rncdl`

CpEcuc_rteEvtMap.rncdl

```
...
task Rte_Dcm_User_Task {
    ...
    # eventMap NameOfInstance          evt_op_req_<NameOfPort>_<NameOfOperation>
    eventMap swcReadWriteDid          evt_op_req_PP_F190_VIN_ReadData
    ...
}
...
```

Before building the PIU image including our swcReadWriteDid, we need to generate sysdesc and bsw using these commands

```
cd ~/device-cm-production

. setenv.sh
clean sysdesc
install sysdesc # Check that you get a similar message in the logs: (e.g. diagnostic-management/1.3.61-pre.2.
g85d73@dev/faceswp-3919-demo-did: Already installed!)
build sysdesc # If this step fails, check that conandata.yml has the correct dependency on the new SWC
clean bsw
install bsw
build bsw
```

(Step 6) - Build the image

The local build is only available on Windows Machines. The best option to build your code is to push your changes to Gitlabee and let the CI build the code for you. Your binary image could be found in the artifacts of the build job.

If you prefer the local build, you can follow the README in the device-cm-production project.